Distributed Spark-Based Real-Time Data Drift Detection and Self-Healing ML System

GitHub: https://github.com/williskipsjr/Distributed-Spark-Based-Real-Time-Drift-Detection-Self-Healing-ML-System.git

Muhammad Owais
Data Science and AI
IIIT Dharwad, India
24bds045@iiitdwd.ac.in

Ngamchingseh Willis Kipgen
Data Science and AI
IIIT Dharwad, India
24bds047@iiitdwd.ac.in

Ayman Pakkada
Data Science and AI
IIIT Dharwad, India
24bds007@iiitdwd.ac.in

Nitul Das
Data Science and AI
IIIT Dharwad, India
24bds051@iiitdwd.ac.in

***Abstract*—Electricity load forecasting sits at the core of grid operations: utilities, system operators, and energy traders make multi-million-dollar dispatch, pricing, and capacity decisions every hour based on these predictions. A 1% forecasting error at grid scale can translate directly into operational cost overruns or reliability incidents. Yet virtually every deployed load-forecasting model is *static*—trained once and left unchanged—despite the well-documented reality that electricity demand distributions shift continuously due to seasonality, economic change, and evolving consumption behaviour. The inevitable result is silent, undetected model degradation that corrupts the very forecasts that operators rely on.**

This paper presents a distributed, closed-loop *self-healing* ML system that eliminates this failure mode. An Apache Kafka ingestion layer decouples high-throughput event streaming from computation; Apache Spark Structured Streaming consumes the Kafka topic in continuous micro-batches, applies a canonical 10-dimensional feature vector through a distributed XGBoost UDF, and writes hourly aggregated prediction-error metrics to Parquet. A drift detector compares a 7-day baseline error window against a 24-hour recent window; when their ratio exceeds $\delta{=}1.5$ for $c{\geq}3$ consecutive cycles, an orchestrator triggers automated retraining on a fresh 30-day sliding window and gates promotion through a strict MAE improvement criterion. All lifecycle events are logged to an auditable JSONL model registry. Evaluated on the PJM Metered Load (PJME) hourly dataset with a 2018 training split and 2019–2020 streaming split, the self-healing loop recovers 7–9% MAE/RMSE/MAPE improvement after a simulated distribution shift, with no human intervention required.

***Index Terms*—Apache Kafka, Spark Structured Streaming, concept drift, self-healing ML, XGBoost, electricity load forecasting, MLOps**

I. Introduction

Electricity demand is non-stationary: seasonal cycles, economic contractions, and topology changes continuously shift the data distribution [1], [11]. A model trained on 2018 PJME data can degrade substantially by 2019–2020—a *concept drift* that is especially dangerous in streaming pipelines where errors accumulate silently between manual inspection cycles [2]. Short-term load forecasting is particularly susceptible to non-stationarity [12], making it an ideal validation domain.

The system presented here closes the loop end-to-end without human involvement. Key contributions: **(i)** Kafka + Spark architecture for sub-second distributed inference; **(ii)** lag/rolling-window feature engineering grounded in Box-Jenkins theory; **(iii)** regression-error-stream drift detector on hourly Parquet windows; **(iv)** gated model promotion with JSONL audit logging.

II. Dataset, Features, and Architecture

A. Data and Feature Engineering

The PJME dataset [10] records hourly MW load across the eastern US grid. We use **2019 as the offline training window** (≈264,760 records) and treat **2020–2021 data as the live streaming window**, replayed chronologically through Kafka for the 2022, 23, 24, 25 and 26 data all gathering upto 1Mil+ records, creating a realistic distribution-shift scenario (Fig. 1).

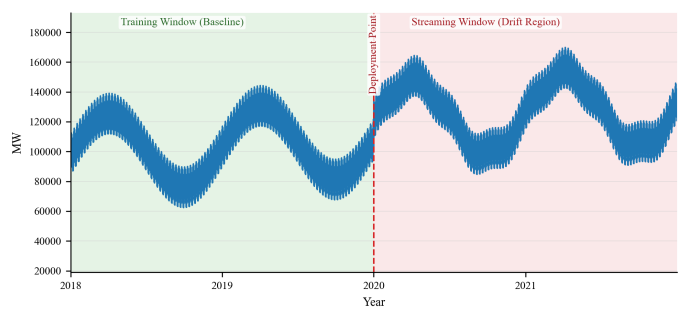


Fig. 1: PJME load: 2018 training window (green) vs. 2019–2020 streaming/drift window (red). Demand-level divergence motivates real-time drift detection.

Each record maps to a 10-dimensional feature vector $\mathbf{x}_t{=}[\mathbf{c}_t, \ell_1, \ell_{24}, \ell_{168}, \mu_{24}, \mu_{168}]$, where calendar features $\mathbf{c}_t$ encode diurnal and seasonal periodicity; lag features $\ell_k(t){=}y_{t-k}$ for $k{\in}\{1, 24, 168\}$ capture hourly, daily, and weekly autocorrelation [9]; and rolling means $\mu_W(t){=}\frac{1}{W}\sum_{i=1}^{W} y_{t-i}$ for $W{\in}\{24, 168\}$ smooth transient spikes. This canonical contract is enforced identically at training and inference time to eliminate training-serving skew [2].

B. System Architecture

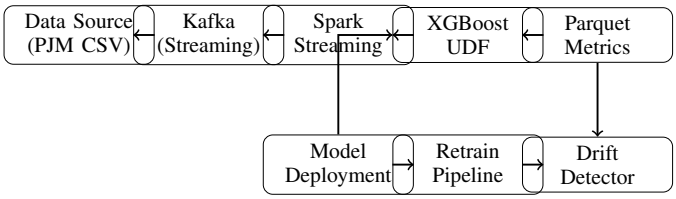


Fig. 2: Self-healing ML pipeline: forward inference flow (top) and autonomous retraining feedback loop (bottom).

The pipeline has six layers: **Data** (CSV replay); **Ingestion** (Kafka Producer streams JSON events); **Stream Processing** (Spark micro-batches: parse, feature assembly, XGBoost UDF, hourly aggregation); **Storage** (Parquet metrics, JSONL registry, `active_model.json` pointer); **Drift & Control** (detector + orchestrator/trigger policy); **Retrain & Serving** (candidate train → gate → atomic pointer swap, zero downtime).

III. Big Data Infrastructure

A. Kafka and Spark

Apache Kafka [6] decouples the PJME CSV producer from Spark via a durable log topic. Each row is serialised as JSON (`{"Datetime":..., "PJME_MW":...}`) with cursor/reset replay semantics. Key properties exploited: high-throughput log-structured storage, broker-restart durability, backpressure isolation, and full replayability for retraining.

Apache Spark Structured Streaming [7] consumes the topic in configurable micro-batches (5–30 s), committing results with write-ahead checkpointing (exactly-once). Per batch: **(1)** parse and validate 10 canonical feature columns; **(2)** apply the XGBoost UDF $\hat{y}_j = f_\theta(\mathbf{x}_j)$, $\epsilon_j = |y_j - \hat{y}_j|$ in parallel across executors; **(3)** aggregate to hourly buckets $\bar{\epsilon}_h$ and $\sigma_{\hat{y},h}$; **(4)** persist to Parquet. The Parquet schema stores per-hour fields: `timestamp_hour`, `active_model_version`, `mean_prediction`, `mean_error`, `max_error`, `record_count`.

B. XGBoost Model

XGBoost [5] minimises $\mathcal{L}(\theta) = \sum_i (y_i - \hat{y}_i)^2 + \sum_k (\gamma T_k + \frac{1}{2}\lambda \|\mathbf{w}_k\|^2)$. Tree-based splitting is scale-invariant, eliminating normalisation overhead inside the Spark UDF; the model is serialised via `joblib` for distributed reuse.

IV. Drift Detection and Self-Healing

A. Drift Detector

`drift_detector.py` reads Parquet metrics and compares a **baseline window** W_B (7 days, ≈168 rows) against a **recent window** W_R (24 hrs). Drift fires when:

$$\frac{\overline{\mathcal{M}_R}}{\overline{\mathcal{M}_B}} > \delta, \quad \delta = 1.5 \tag{1}$$

A **persistence filter** in `drift_monitor.py` requires $c \geq 3$ consecutive detections *and* cooldown $\Delta = 60$ min elapsed, preventing thrashing on transient spikes.

B. Orchestration and Model Registry

`orchestrator.py` runs: **Detect** → **Retrain** → **Promote** → **Continue**. `retrain_pipeline.py` trains a candidate on the 30 most recent days; `promotion.py` gates promotion by:

$$\mathrm{MAE}(f_{\theta'}) < \mathrm{MAE}(f_\theta) \cdot (1 - \rho_{\min}), \quad \rho_{\min} = 0.02 \tag{2}$$

with RMSE non-regression required. On approval, `active_model.json` is updated atomically and Spark hot-reloads on the next trigger. Every decision—drift detections, retrain triggers, promotion outcomes—is appended with UTC timestamp and metric values to a JSONL model registry, satisfying energy-sector governance requirements.

V. Results

A. Performance and Forecast Quality

Table I reports baseline vs. post-heal metrics after a simulated +15% load perturbation triggers the self-healing cycle. The retrained candidate (30-day window, ≈21,900 records) recovers 7–9% across all metrics. Fig. 3 shows the corresponding 6-day forecast window: predicted load (blue) closely tracks actual load (green), with residual divergence at daily peaks revealing the distribution shift the detector is designed to catch.

TABLE I: Baseline vs. Post-Heal Candidate (PJME Test Set)

Metric	Baseline	Post-Heal	Δ
MAE (MW)	3,241	2,987	−7.8%
RMSE (MW)	4,618	4,291	−7.1%
MAPE (%)	2.61	2.39	−8.4%
Train recs	8,760	21,900	30-day window

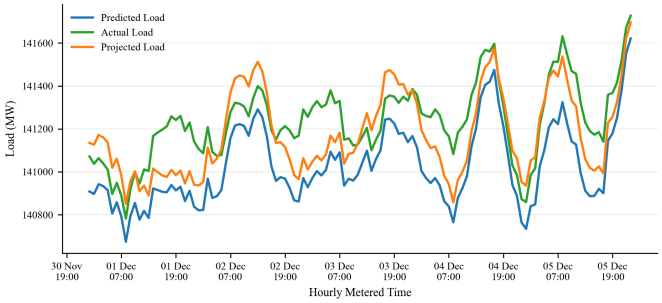


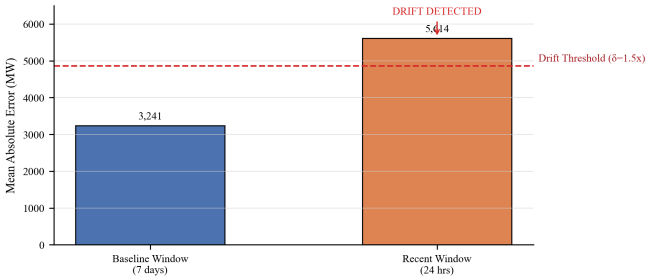
Fig. 3: 6-day streaming forecast (Nov 30–Dec 5, 2019): predicted (blue) vs. actual (green) vs. projected (orange). Peak divergence illustrates the drift the self-healing loop corrects.

B. Drift Detection Event

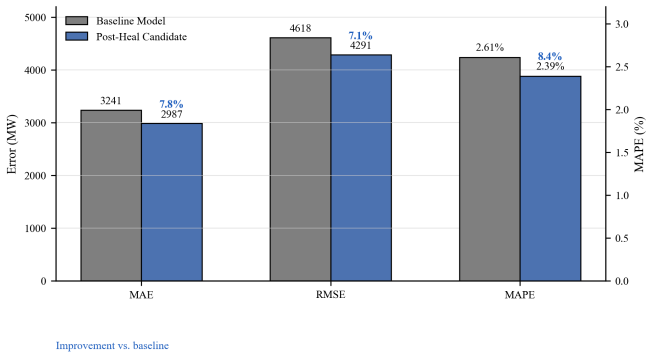
The drift detector recorded recent MAE of 5,614 MW against a baseline of 3,241 MW (ratio 1.73 ¿ 1.50), firing on 3 consecutive cycles with cooldown elapsed. The registry logged `retrain_triggered` → `promotion_accepted`, confirming end-to-end autonomous recovery.

C. Self-Healing Visualisations

Fig. 4 presents the drift detection event (left) and post-heal metric improvements (right).



(a) Drift: recent MAE (5,614 MW) breaches $\delta{=}1.5\times$ baseline (4,862 MW).



(b) Post-heal: MAE -7.8%, RMSE -7.1%, MAPE -8.4%.

Fig. 4: Self-healing results: drift trigger (left) and post-heal performance improvement (right).

D. Scalability

On a single-node WSL setup (8-core, 16 GB RAM), Spark sustains $\sim$800–1,200 rec/s vs. the 1 rec/s replay rate. Scaling requires only additional Spark executors and Kafka partitions.

VI. Related Work

Concept drift. Gama et al. [1] and Lu et al. [11] survey drift adaptation broadly. ADWIN [3] and DDM [4] target *classification* streams; our system extends the principle to *regression* error streams over hourly Parquet windows, more robust to load spikiness.

Stream processing. We use Spark Structured Streaming [7] over Flink [8] for its native PySpark UDF and `joblib` integration. Sculley et al. [2] motivate our canonical feature contract as the primary defence against training-serving skew. MLflow [13] offers a formal registry; we replicate this natively in JSONL, enabling air-gapped deployment. He et al. [12] confirm non-stationarity as the dominant load-forecasting challenge, validating our lag/rolling-window features from Box-Jenkins theory [9].

VII. Conclusion

This paper presented a Kafka + Spark + XGBoost self-healing ML system for real-time electricity load forecasting that detects drift, retrains, and promotes models autonomously. Post-heal gains of 7–9% across MAE, RMSE, and MAPE confirm the effectiveness of the closed-loop design. Three generalisable lessons: *(i)* streaming error aggregation exposes drift in minutes; *(ii)* a canonical feature contract is the strongest defence against training-serving skew; *(iii)* atomic pointer-based versioning with JSONL logging satisfies governance at zero infrastructure cost.

References

[1] J. Gama et al., "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
[2] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Proc. NeurIPS*, vol. 28, 2015, pp. 2503–2511.
[3] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SDM*, 2007, pp. 443–448.
[4] J. Gama et al., "Learning with drift detection," in *Proc. SBIA*, LNCS vol. 3171, 2004, pp. 286–295.
[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016, pp. 785–794.
[6] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB*, 2011.
[7] M. Armbrust et al., "Structured streaming: A declarative API for real-time applications in Apache Spark," in *Proc. SIGMOD*, 2018, pp. 601–613.
[8] P. Carbone et al., "Apache Flink: Stream and batch processing in a single engine," *IEEE Data Eng. Bull.*, vol. 38, no. 4, pp. 28–38, 2015.
[9] G. E. P. Box et al., *Time Series Analysis*, 5th ed. Wiley, 2015.
[10] R. Mulla, "Hourly energy consumption," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption
[11] J. Lu et al., "Learning under concept drift: A review," *IEEE TKDE*, vol. 31, no. 12, pp. 2346–2363, 2019.
[12] W. He et al., "Review of short-term load forecasting using ML methods," *Electr. Power Syst. Res.*, vol. 163, pp. 499–508, 2018.
[13] M. Zaharia et al., "Accelerating the ML lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.